

JAS OS

Technical Report

Implementing and proving my understanding of CSE 323 lecture concepts

Student	Jubayed Ahmed Sahel
Student ID	2221173642
Course	CSE 323, Section 3
Project version	JAS OS v1.5.4
Scope	Main lecture concepts from Chapters 1-13; Chapter 14 onward excluded
Project motive	Implement, test, and prove my own understanding — not teach other people

github.com/Jubayed-Sahel/JAS-OS

Report structure: objective and scope · implemented concepts · evidence · STAR challenges · verification · submission links

1. Executive Summary

JAS OS is a bootable, freestanding 32-bit x86 course-project operating system. Its purpose is to implement, test, and prove my own understanding of the main CSE 323 lecture concepts from Chapters 1-13. It is not intended to teach other people. Each feature connects a lecture concept to code, changing kernel state, and an observable algorithmic result.

2. Project Objective and Scope

The previous project was an ESP32-S3 mini-kernel. My objective was to transfer its useful operating-system ideas to a PC-style x86 kernel that boots from an ISO in Oracle VirtualBox. I included the main concepts through Chapter 13 and excluded Chapter 14 onward as required.

Included. A BIOS boot path, protected-mode kernel, graphical desktop, terminal, Files, Notes, Task Manager, Calculator, Settings, Clock, OS Lab, and observable implementations for scheduling, synchronization, deadlocks, memory, paging, file systems, storage, and I/O.

Excluded. Chapter 14 onward, networking, ring-3 isolation, production device drivers, and claims of production persistence or security.

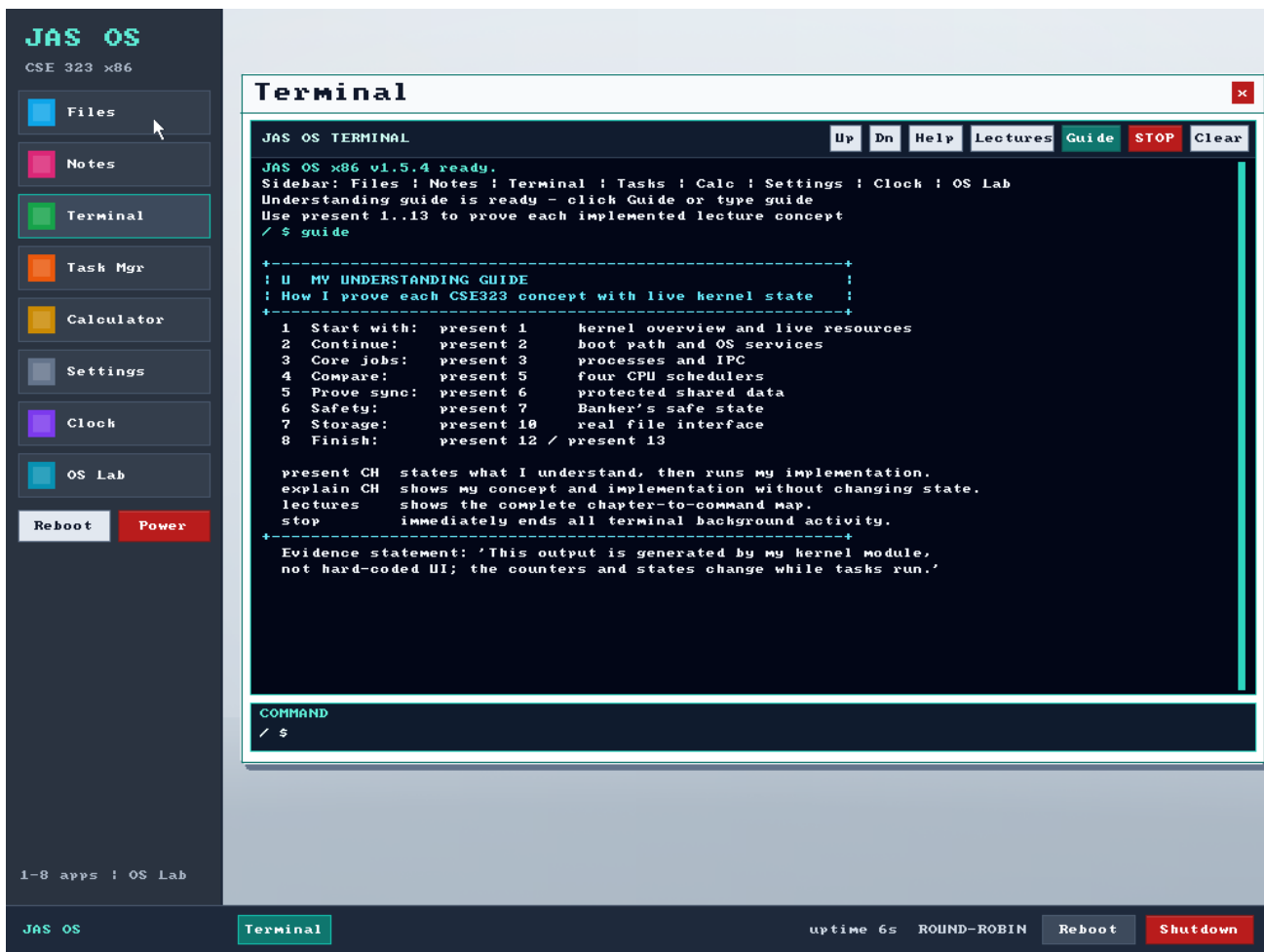


Figure 1. JAS OS understanding guide running from the rebuilt ISO in Oracle VirtualBox.

3. System Architecture

The BIOS loads a custom El Torito boot image. A 16-bit loader reads the kernel, requests a VESA framebuffer, enables A20, installs the GDT, and enters 32-bit protected mode. The kernel initializes IDT/PIC/PIT, serial, PS/2 input, graphics, tasks, scheduling, memory, paging, synchronization, Banker's algorithm, MiniFS, storage/I/O models, event logging, and the GUI.

Boot and hardware	BIOS/El Torito, VESA, A20, GDT, IDT, PIC, PIT, PS/2
Core execution	Task control blocks, cooperative scheduler, semaphores, event log
Memory and safety	First-fit heap, paging model, FIFO replacement, Banker's algorithm
Storage and I/O	MiniFS, disk scheduling, RAID model, polling, interrupts, DMA
Evidence interface	Desktop, Terminal, understanding Guide, OS Lab, global STOP

4. Implemented Lecture Concepts

Chapter	Lecture concept	Evidence in JAS OS
1	OS organization	Live subsystem dashboard
2	Services and boot	Boot trace and syscall aliases
3	Processes and IPC	Task states and mailbox send/receive
4	Threads	Shared-counter workers
5	CPU scheduling	RR, Priority, FCFS, SJF, timeline
6	Synchronization	Semaphores and readers-writers
7	Deadlocks	Banker's safety algorithm
8	Main memory	First-fit heap and coalescing
9	Virtual memory	Paging, faults, frames, FIFO
10	File-system interface	MiniFS directories and files
11	File-system implementation	Allocation-method comparison
12	Mass storage	Five disk algorithms and RAID model
13	I/O systems	Polling, interrupts, DMA counters

5. Evidence of My Understanding

The explain CH command states the concept and identifies my implementation. The present CH command then runs that implementation and prints live evidence. Scheduling changes the CPU timeline; readers-writers changes semaphore-protected state; Banker's algorithm calculates a safe sequence; and paging changes frame ownership and fault counts. OS Lab calls the same evidence path with buttons. The global stop ends all non-shell activity while keeping the terminal usable.

6. Challenges Using the STAR Format

Challenge 1 — Moving from ESP32-S3 to a Bootable x86 ISO

SITUATION	The previous mini-kernel depended on an ESP32-S3 runtime, while this project needed to boot as a PC operating system in Oracle VirtualBox.
TASK	Preserve the relevant course concepts without a host operating system or standard library and provide keyboard, mouse, and graphical output.
ACTION	I implemented a BIOS/El Torito loader, VESA framebuffer setup, A20, GDT and protected-mode transition, then added IDT/PIC/PIT, serial, PS/2 input, graphics, and freestanding runtime modules.
RESULT	JAS OS boots from its own ISO into an interactive 1024×768 desktop, and the cross-compiler build completes without warnings.

Challenge 2 — Proving Understanding Instead of Printing Labels

SITUATION	Printing algorithm names would not demonstrate that I understand how the concepts operate or affect system state.
TASK	Connect every included chapter to stateful code and observable results.
ACTION	I separated scheduling, tasks, synchronization, memory, paging, Banker's algorithm, files, storage, and I/O into modules and exposed their changing state in the terminal.
RESULT	Chapters 1-13 each have a command and OS Lab button backed by live kernel state, allowing me to explain what changed and why.

6. Challenges Using the STAR Format — Continued

Challenge 3 — Organizing Evidence During Assessment

SITUATION	The command set became difficult to remember, and background output could hide the result I needed to discuss.
TASK	Make my implementation evidence easy to find without replacing the algorithms with static interface text.
ACTION	I added the understanding Guide, explain CH, present CH, chapter map, readable terminal layout, scrolling controls, and OS Lab shortcuts. Each path names the concept, my implementation, and its live proof.
RESULT	I can follow a consistent assessment route and connect each lecture concept to the corresponding source-backed result.

Challenge 4 — Stopping Every Background Task Safely

SITUATION	Producer-consumer, thread, readers-writers, and counter jobs can generate asynchronous terminal output. The old stop action controlled only one demonstration.
TASK	Stop all background activity without terminating the shell.
ACTION	The global stop handler calls each lab's shutdown, scans the task table, terminates remaining non-shell tasks, and restores scheduler state.
RESULT	stop, stop all, lab stop, and the red STOP button end terminal background activity while the shell remains ready.

7. Verification and Result

Check	Verified result
Kernel	93,408 bytes; build completed with no compiler warnings
ISO	1,523,712 bytes
ISO SHA-256	CC89E64B14337AFAE5DAEE51B2C7A059C92BE3D808551495B41C981EAF17A2C1
Virtual machine	32-bit BIOS, 1024×768 framebuffer, PS/2 input
Lecture scope	Main concepts from Chapters 1-13; Chapter 14 onward excluded
Global stop	Ends non-shell activity and leaves the terminal responsive
Demonstration	4:20 narrated 1080p MP4 using genuine JAS OS captures
Silent demo	4:00 terminal typing and guest-cursor MP4 with matching voice-over script

8. Submission Links

GitHub repository: <https://github.com/Jubayed-Sahel/JAS-OS>

Bootable ISO: <https://github.com/Jubayed-Sahel/JAS-OS/blob/main/jas-os.iso>

Narrated demonstration: <https://github.com/Jubayed-Sahel/JAS-OS/blob/main/docs/JAS-OS-demo.mp4>

Silent terminal demonstration: <https://github.com/Jubayed-Sahel/JAS-OS/blob/main/docs/JAS-OS-Silent-Terminal-Demo.mp4>

9. Limitations and Scope Boundary

JAS OS is a course-project kernel, not a production operating system. Tasks are cooperative, applications execute in kernel space, and MiniFS is RAM-backed. Paging, disk, RAID, and I/O are transparent models used to test my understanding. The project does not claim ring-3 isolation, production device drivers, networking, persistence, or Chapter 14+ protection and security.

Conclusion

The completed JAS OS package demonstrates that I can connect the main CSE 323 lecture concepts to a bootable implementation, inspect their state, explain the algorithms, and verify their results. The project motive is evidence of my own understanding, and the OS, report, repository, and narrated demonstration now state that consistently.